



Descrição do Segundo Trabalho

Objetivo

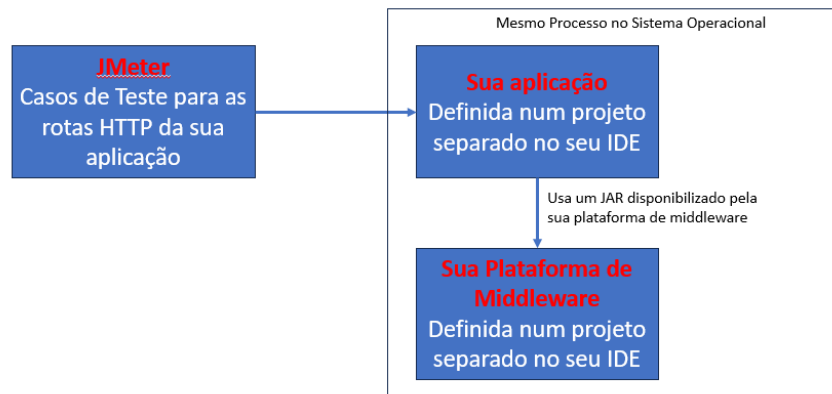
Desenvolver uma Plataforma de Middleware em Java implementada por meio do uso dos Padrões de Projeto descritos no livro "**Remoting Patterns: Foundations of Enterprise, Internet and Realtime Distributed Object Middleware**". Instanciar a plataforma proposta implementando a mesma aplicação do trabalho da primeira unidade.

Requisitos

1. Trabalho individual.
2. A plataforma de middleware desenvolvida deve possuir um Modelo de Componente capaz de definir um objeto por meio de anotações e viabilizar a execução dos métodos desse objeto remoto por meio de invocações de requisições HTTP.
3. Cada aluno(a) deve implementar os seguintes padrões na sua respectiva plataforma:
 - Broker, Basic Remoting Patterns(Server Request Handler, Invoker, Marshaller, Remote Object, Remoting Error), Identification Patterns (Lookup, Object Id, Absolute Object Reference), Lifecycle Management(Static Instance, Per-Request Instance, Client-Dependent Instance)[2 de 3], Lifecycle Management(Lazy Acquisition, Pooling, Leasing, Passivation)[2 de 4], Extension Patterns(Invocation Interceptor, Invocation Context, Protocol Plug-In(deve-se suportar dois protocolos. Exe.: UDP e TCP)).
4. Testes de Carga com JMeter
 - Deve-se utilizar a ferramenta JMeter para realizar testes de carga da aplicação desenvolvida utilizando a Plataforma de Middleware proposta.
 - No dia da apresentação do projeto, todos os testes devem estar preparados e configurados antecipadamente.
5. Deve-se investigar a capacidade do sistema, e conhecer seu **Knee capacity** e **Usable Capacity**

Abaixo, segue uma figura mostrando uma configuração mínima do sistema distribuído a ser apresentado. Note que é preciso criar um projeto para a sua aplicação separado do projeto da sua plataforma de middleware. O projeto deve importar uma biblioteca (.JAR) que contenha sua plataforma de middleware para que a mesma seja instanciada e viabilize a

execução da aplicação. Por sua vez, o JMeter deve ser utilizado para enviar requisições HTTP para o endereço IP/Porta definido pela sua aplicação.



Fluxo de Execução da Arquitetura

Nesta arquitetura, o **JMeter** será responsável por enviar requisições para a sua aplicação. Essas requisições devem ser feitas utilizando os samplers **HTTP** ou **outros** disponíveis para interação com sua plataforma de middleware. Deve-se implementar a mesma aplicação desenvolvida no trabalho da primeira unidade. Neste caso, as classes de negócio devem ser anotadas com as anotações definidas pela sua plataforma de Middleware.

CrITÉRIOS de Avaliação:

- **Individual:** Modelo de Componentes (2,0), Basic Remoting Patterns(2,0), Identification Patterns(1,0), Lifecycle Management Patterns(2,0), Extension Patterns(3,0).
- O **Modelo de componentes** se refere a como seu objeto remoto é descrito. Abaixo temos dois exemplos de anotações que descrevem um método remoto: O primeiro é exemplo é mais simples, pois tudo é passado via JSONObjects. No segundo, é descrito um modelo mais sofisticado onde a assinatura do método é preservada. Quanto mais sofisticado maior a nota para esse critério:

```
1. @Post(router = "/sub")
   public JSONObject sub(JSONObject jsonObject) throws Throwable {

2.     @RequestMapping(method = MethodHTTP.GET, path = "soma")
       public int soma(@Param(name = "a") int num1, @Param(name = "b") int num2) {
           return num1 + num2;
       }
}
```

- A **Apresentação** se refere ao momento em que o(a) aluno(a) irá apresentar o trabalho para o restante da turma. Neste momento, a solução precisa funcionar com execução de testes via JMeter para todos os padrões implementados.